



# MTA

## AUDIO EDITOR

### Manuale Amministratore

Deployment, sicurezza, storage e operations

**Versione: 0.2.0 · Revision: 159**

**Build: 20261006021358**

Alessandro De Salvo

braket71@gmail.com

<https://github.com/desalvo/mta-audio-editor>

Licenza: EUPL-1.2

Digital Audio Workstation con supporto MTA



Windows



macOS



Linux



Android



iOS

Guida operativa per installazione, sicurezza, storage, backup, client desktop/mobile, pipeline audio, formato MTA, osservabilità e release.

## Indice

1. Architettura
2. Deployment
3. Configurazione
4. Storage e backup
5. Sicurezza e utenti
6. Pipeline audio
7. Client desktop e mobili
8. Formato MTA
9. Operazioni e osservabilità
10. Upgrade e rollback
11. CI/CD e release
12. Troubleshooting amministrativo

## 1. Architettura

MTA Audio Editor usa un backend FastAPI e un frontend web condiviso da browser, client desktop e app mobili. Rendering, waveform, Demucs e generazione MTA sono operazioni backend.



Figura 1 - Client, API, processing e storage.

| Componente | Piattaforme                               |
|------------|-------------------------------------------|
| Server     | Linux, Docker, Kubernetes                 |
| Desktop    | Windows, macOS Intel, macOS Apple Silicon |
| Mobile     | Android, iOS/iPadOS                       |
| Browser    | Browser moderni con Web Audio API         |

## 2. Deployment

Per Docker Compose usare un volume persistente per /data/projects. In Kubernetes usare il PVC previsto dal manifest e un Ingress opzionale. Il limite upload applicativo e quello dell'Ingress devono essere coerenti.

```
docker compose up -d
# Kubernetes
kubectl apply -k k8s/overlays/nginx
```

**TLS:** in produzione terminare HTTPS su un reverse proxy/Ingress affidabile. Configurare una URL pubblica coerente per link email e OAuth quando utilizzati.

## 3. Configurazione

Le impostazioni principali sono fornite tramite variabili d'ambiente e Secret. Tra i gruppi da gestire: bootstrap amministratore, upload massimo, SMTP, OAuth, sicurezza sessione e storage.

| Area          | Esempi                                                        | Nota                                       |
|---------------|---------------------------------------------------------------|--------------------------------------------|
| Bootstrap     | MTA_ADMIN_USERNAME,<br>MTA_ADMIN_PASSWORD,<br>MTA_ADMIN_EMAIL | Necessari per il primo amministratore      |
| Upload        | MTA_MAX_UPLOAD_MB                                             | Allineare Ingress e applicazione           |
| Pubblicazione | MTA_PUBLIC_URL                                                | Consigliato dietro proxy per callback/link |
| SMTP          | host, porta, TLS, credenziali                                 | Secret, non ConfigMap pubblica             |

## 4. Storage e backup

Il root persistente contiene database utenti/configurazione e directory progetto. Ogni progetto mantiene modello, audio, attachment, originali, cache e analisi. I backup devono includere l'intero root persistente quando si vuole un ripristino completo.

**Strategia:** snapshot coerente del volume, retention su più generazioni e test periodico di restore. Per Kubernetes verificare che StorageClass e CSI supportino correttamente UID/GID e fsGroup usati dal pod.

## 5. Sicurezza e utenti

Le installazioni server supportano account locali, verifica email, approvazione amministrativa, TOTP e provider OAuth opzionali. Le funzioni social sono disabilitate per default. Le richieste OAuth sono vincolate a HTTPS e host provider consentiti.

La modalità desktop native single-user evita database account e schermate login, ma mantiene i controlli di accesso interni necessari alle API locali. Le credenziali e i token non devono essere incorporati nelle immagini container.

## 6. Pipeline audio



Editing non distruttivo: le sorgenti restano intatte fino al rendering finale.

Figura 2 - Editing non distruttivo e rendering.

Le operazioni FFmpeg e Demucs possono essere CPU/GPU intensive. Le preview traccia vengono cacheate con una firma che include sorgente, stato traccia e parametri rilevanti. I job lunghi espongono avanzamento e risultato tramite API.

### 6.1 Preview dinamica e Render

La preview dinamica bufferizza le tracce sul client e mantiene un clock di sincronizzazione. Render genera un master server-side. Con i meter RealTime, un ramo monitor delle tracce alimenta gli analyser senza diventare udibile.

## 7. Client desktop e mobili

I desktop includono backend locale e selettori filesystem nativi. Le app mobili sono client del backend remoto.

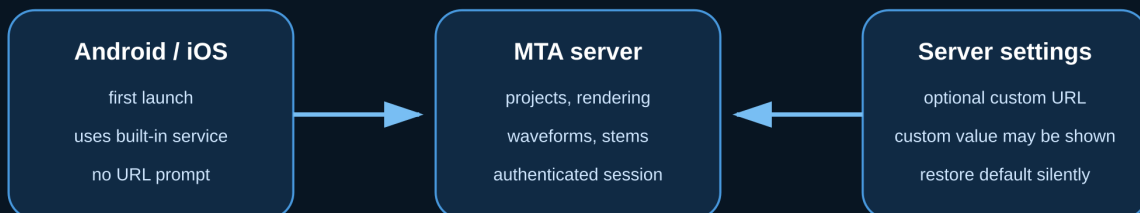
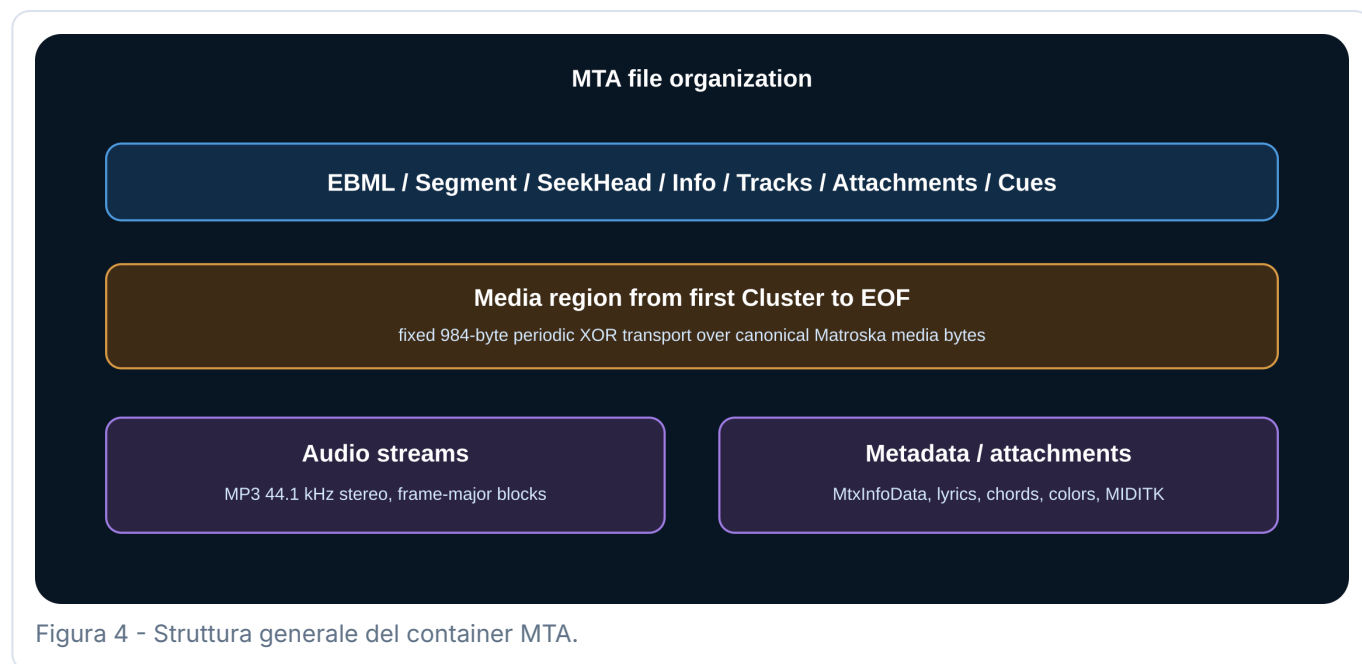


Figura 3 - Connessione mobile.

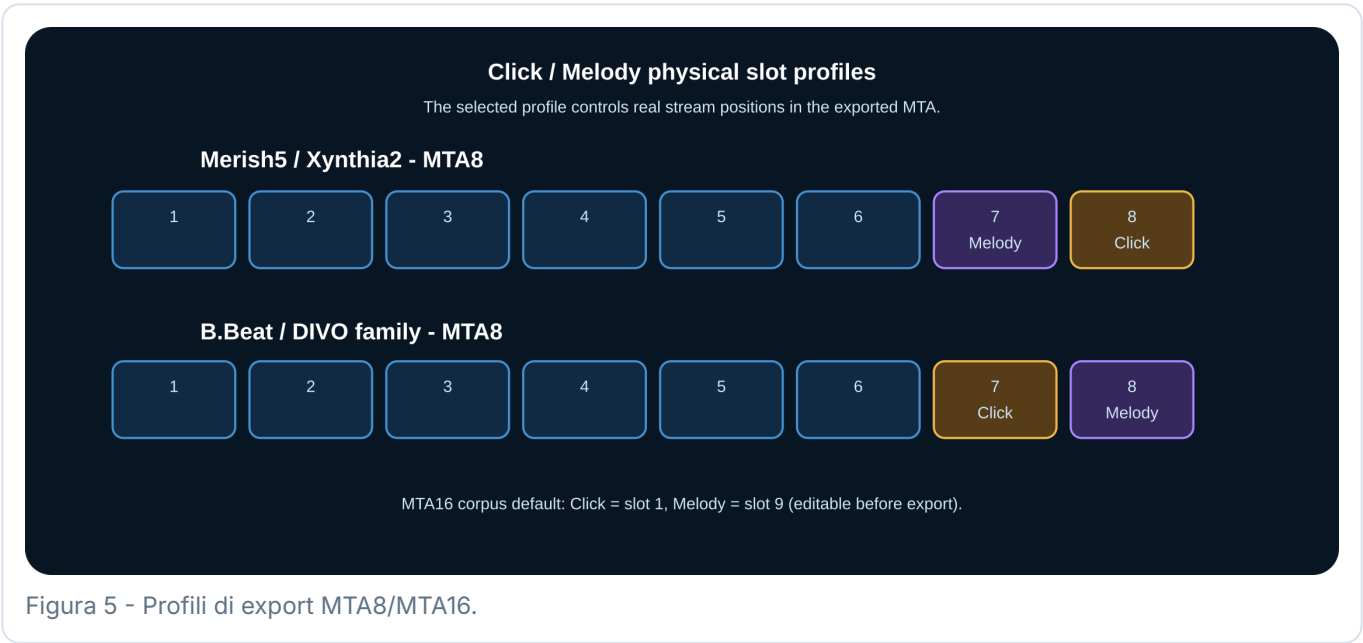
Android/iOS usano automaticamente il servizio preconfigurato senza prompt durante installazione/primo avvio. L'indirizzo predefinito resta interno e non viene mostrato. Un URL personalizzato può essere configurato successivamente e, poiché è scelto dall'utente, può essere mostrato nelle impostazioni.

## 8. Formato MTA



MTA usa strutture EBML/Matroska, stream audio MP3 e attachment di metadati. La regione media a partire dal primo Cluster usa un trasporto XOR periodico documentato. La specifica completa IT/EN è in docs/MTA\_FORMAT\_FINAL\_SPEC\_IT.md e docs/MTA\_FORMAT\_FINAL\_SPEC\_EN.md.

## 8.1 Profili Click/Melody



I profili MTA8 preservano posizioni fisiche specifiche. Il default MTA16 basato sul corpus usa Click 1 e Melody 9, sempre modificabili. Gli slot silenziosi intermedi vengono creati quando necessari per mantenere il numero fisico dello stream.

## 9. Operazioni e osservabilità

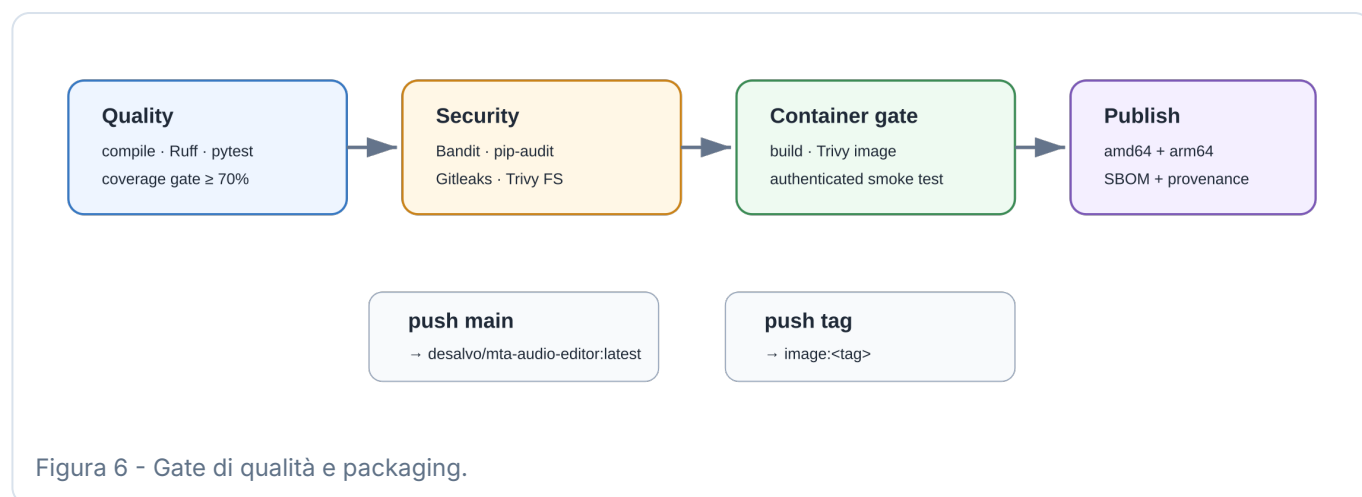
Monitorare health endpoint, spazio persistente, tempi di render, job lunghi, log applicativi, FFmpeg/Demucs e disponibilità del servizio. Le probe Kubernetes devono rimanere leggere e indipendenti dai job audio.

| Segnale | Controllo                                   |
|---------|---------------------------------------------|
| Health  | /api/health                                 |
| Storage | Spazio libero e inode del volume            |
| Job     | Durata, stato, errori server-side           |
| Auth    | Errori login/OAuth/SMTP                     |
| Media   | ffmpeg, Demucs, generazione waveform/export |

## 10. Upgrade e rollback

Prima dell'upgrade effettuare backup, verificare release notes e production gate, aggiornare immagini/manifests e validare l'apertura di progetti esistenti. Per rollback ripristinare la versione applicativa e, se necessario, lo snapshot del volume coerente con quella versione.

## 11. CI/CD e release



Il production gate include lint, test, controlli sicurezza e packaging. Le build mobile e native sono prodotte da workflow separati/step dedicati; i secret di signing restano nel sistema CI.

**Identità di release:** VERSION contiene la versione pubblica del prodotto (es. 0.2.0), REVISION identifica la revisione esatta del sorgente/package (es. 153) e BUILD\_INFO resta l'identificativo di build indipendente a 14 cifre. I tag stabili usano v0.2.0; gli artefatti possono includere -r153. Gli eventuali *versionCode/CFBundleVersion* richiesti dalle piattaforme sono codici tecnici di packaging e non sostituiscono BUILD\_INFO.

## 12. Troubleshooting amministrativo

| Sintomo                                 | Verifica                                                     |
|-----------------------------------------|--------------------------------------------------------------|
| Upload 413/limitato                     | Allineare limite app e proxy/Ingress                         |
| Job stems molto lento                   | CPU/GPU, RAM, spazio temporaneo, modello Demucs              |
| Login/email non funzionano              | SMTP, URL pubblica, log auth, stato utente                   |
| Pod non scrive sul PVC                  | permessi storage, UID/GID, fsGroup                           |
| Client mobile non collega server custom | URL custom, TLS, DNS e raggiungibilità dal dispositivo       |
| MTA non accettato dall'hardware         | profilo slot, target, firmware e validazione sul dispositivo |

## Demucs locale iPhone/iPad - provisioning Core ML

Il server può distribuire modelli Demucs compatibili con Core ML impostando `MTA_DEMUCS_COREML_MODEL_DIR` su una directory contenente `demucs-2.mlmodel`, `demucs-4.mlmodel`, `demucs-6.mlmodel` e/o `demucs-8.mlmodel`. Il client iOS/iPadOS scarica il modello autenticandosi con la sessione corrente, lo compila localmente e lo conserva in Application Support. Le separazioni successive possono funzionare offline sul dispositivo; in modalità Auto il server resta il fallback per modelli mancanti, brani oltre il limite locale o errori di inferenza.

## Aggiornamenti nativi e modelli Demucs

Le applicazioni native Windows, macOS, Android, iPhone e iPad controllano GitHub per nuove versioni. Il canale **Stable** usa solo release/tag, mentre **Early release** include anche la prerelease `early-main` prodotta dall'ultima build valida di main.

Su iPhone/iPad l'app rende disponibile automaticamente un modello Demucs Core ML 4-stem predefinito: usa prima l'eventuale modello bundled e, se assente, lo scarica dal server configurato. Gli hash SHA-256 vengono poi confrontati con il catalogo server e i modelli installati vengono aggiornati quando cambia il fingerprint. Il modello locale già installato resta utilizzabile offline.

Su iOS/iPadOS l'aggiornamento dell'app deve comunque passare da TestFlight/App Store o da un canale di distribuzione autorizzato dal sistema operativo.

## Aggiornamento automatico modelli Demucs mobile

---

Il server Docker/Kubernetes aggiorna periodicamente i repository Core ML (iPhone/iPad) e ONNX (Android) da un manifest HTTPS, verificando SHA-256 e sostituendo i file in modo atomico. I client mobili controllano il catalogo ogni 6 ore e all'avvio. La preferenza **Update solo con Wi-Fi** è attiva per default: gli aggiornamenti periodici dei modelli vengono scaricati solo su Wi-Fi, salvo disabilitazione esplicita dello switch. Il modello baseline 4-stem viene invece sempre predisposto alla prima occasione di rete disponibile.

## Registro modelli stem esteso

---

Configurare modelli server aggiuntivi con `MTA_DEMUCS_MODEL_REGISTRY` o `MTA_DEMUCS_MODEL_REGISTRY_FILE`. Ogni profilo dichiara `model`, `stem_count`, `stem_labels` e opzionalmente `engine/display_name`. I repository Core ML/ONNX vengono scoperti dinamicamente tramite file `demucs-N`, fino a 64 stem.

## Gestione dinamica dei modelli Demucs

---

Il server mantiene periodicamente aggiornato l'intero catalogo di modelli configurato, inclusi i modelli Demucs ufficiali disponibili nel runtime e gli artefatti Core ML/ONNX pubblicati dal manifest. I client mobile e desktop scaricano on-demand il modello scelto quando viene richiesto lo splitting. Dalle opzioni locali è possibile cancellare un modello, riscarlo o forzarne l'aggiornamento. L'amministratore può blacklistare un modello: il modello viene rimosso dallo storage gestito e scompare dal catalogo disponibile a web e client finché non viene rimosso dalla blacklist.

## Token runtime per i modelli

---

`POST /api/models/token` emette token Bearer revocabili con solo scope `models:read`. I download Core ML, ONNX e bundle nativi richiedono sessione autenticata o tale token. La CI usa asset della release GitHub `demucs-models` e non credenziali runtime.

## Motori Lyrics/Chords e MP4

---

Per la massima accuratezza delle lyrics, il default è `MTA_LYRICS_WHISPER_MODEL=large-v3`; è configurabile anche `MTA_LYRICS_BEAM_SIZE`; con temperatura 0 viene usata una ricerca beam deterministica con patience aumentata. Per i chords, se Sonic Annotator con plugin Chordino/NNLS-Chroma è installato viene preferito automaticamente. L'export MP4 richiede FFmpeg con filtro `ass/libass` e codec H.264/AAC.

## Integrazione repertori SIAE / Soundreef

---

SIAE e Soundreef sono i provider attivi per default nella ricerca repertorio. Le pagine pubbliche ufficiali restano sempre disponibili come fallback. Per una ricerca strutturata server-to-server configurare endpoint HTTPS autorizzati/licenziati con `MTA_RIGHTS_SIAE_SEARCH_URL` e/o `MTA_RIGHTS_SOUNDREEF_SEARCH_URL`. L'endpoint deve restituire JSON con una lista in `results`, `items`, `works` o `data`; vengono riconosciuti campi comuni quali `title/original_title`, `authors/writers/composers`, `performers/artists`, `publishers/editors`, `ISWC`, `IPI` e `work_id`. L'app non presume endpoint privati o non documentati delle collecting.